

Conditional Trajectory Abstraction for Visual Action Ranking

Anonymous research manuscript

Abstract

Selecting an action chunk from a visual policy requires predicting consequences that distinguish closely related proposals. Predicting spatial features retains a large visual target, whereas directly regressing a score provides no explicit future representation. We investigate Conditional Trajectory Abstraction (CTA), a learned prediction target for this decision: a source encoder compresses sampled future observations jointly with observed context; a world model forecasts that code from context and an action chunk; and a goal-conditioned reader scores the forecast without access to proposed actions. Ranking and feature-reconstruction supervision shape the source, followed by categorical prediction and reader-score consistency. Deployment uses 48 continuous expected-quantization coordinates, rather than future images. On identical eight-candidate PushT development banks, CTA recovers 0.680 of the available geometry improvement, compared with 0.635 for endpoint prediction and 0.502 for a parameter-matched direct scorer; paired 95% intervals support both ranking differences. Historical closed-loop results are less decisive: CTA succeeds in 140/200 episodes versus 138/200 for endpoint prediction, and proposal batch sensitivity limits reproducibility. Longer-horizon diagnostics also expose a source-to-prediction gap. These findings support compact future-supervised targets for action ranking while identifying prediction fidelity, native-objective alignment, and reproducible held-out control as the outstanding tests of their practical value.

1. Introduction

A visual policy can propose several plausible action chunks, yet small differences between them can decide whether an object reaches its target. Sampling additional proposals helps only when a selector can identify the useful ones. This makes the prediction target a practical design choice: what consequence should a world model forecast so that a reader can rank actions accurately and economically?

Existing approaches offer different answers. Visual world models predict future spatial features or observations [2, 16]; these targets support image-goal evaluation but retain sub-

stantial spatial structure. Compact visual tokenizers reduce that representation [8]. Direct value guidance instead scores candidate actions [11], while predictive policy steering evaluates their generated consequences [4, 13]. These methods establish that prediction, compression, and proposal selection can each benefit control. They leave a focused empirical question: how useful is a learned, context-conditioned *segment target* between a full visual forecast and a direct score, when proposals and evaluation states are held fixed?

We study this question with Conditional Trajectory Abstraction (CTA). During training, an encoder observes context C and sampled future observations τ , producing a compact code S . A predictor learns $p(S \mid C, \mathbf{A})$ for a proposed action chunk $\mathbf{A}$. At deployment, a reader evaluates the prediction together with C and a goal image g . Its only candidate-specific input is the predicted code: it cannot bypass the representation by reading $\mathbf{A}$. Actual future observations are training targets, never deployment inputs. Context conditioning permits the source and reader to share information, and segment supervision includes intermediate observations; the benefits of those individual choices still require isolated ablations.

This design is related to conditional trajectory codes [7], context-aware tokenization [10], and decision-aware model compression [1, 5]. Our methodological contribution is their concrete use as a future-supervised target for visual action ranking: the code is learned from observed consequences, forecast from a supplied action chunk, and evaluated through an action-blind reader. The contribution concerns this target-learning interface and its decision quality, building on those established ingredients.

We ask three questions. **RQ1:** Does this target improve ranking on identical candidate banks relative to endpoint prediction and direct scoring? **RQ2:** Does ranking quality translate into closed-loop success and useful computation savings? **RQ3:** Which failures arise in source encoding, prediction, or the evaluation protocol? The answers are deliberately separated. CTA improves development-set geometry ranking, including against a larger direct scorer. Its historical closed-loop advantage over learned alternatives is much less certain. Component timing shows a compact scoring path, but policy sampling dominates and total-planner

savings are not established. Longer-horizon prediction errors and sensitivity to proposal batch shape explain concrete limits of the current evidence.

The paper contributes: (1) a conditional segment-target architecture with an explicit deployment information boundary; (2) a two-stage training recipe combining future-feature reconstruction, candidate ranking, and forecast-to-reader consistency; and (3) a development evaluation separating privileged future reading, predicted-code ranking, historical closed-loop behavior, and numerical reproducibility. This third contribution includes negative findings: a lower prediction loss can accompany code collapse, and a ranking gain need not produce a reliable episode-level gain. The goal is to establish what this representation currently buys and what evidence is still needed for a general quality–compute claim.

2. Related Work and Positioning

Visual prediction and compact representations. DINO-WM predicts frozen DINOv2 spatial features for image-goal planning [12, 16]; V-JEPA 2-AC predicts features of its own frozen video encoder [2]. Keeping a spatial target offers broad visual supervision, but the target need not be minimal for a particular decision. CompACT learns compact discrete *observation* tokens and demonstrates a planning/compute tradeoff [8]. CTA instead learns one context-conditioned code for sampled observations spanning an action segment. Our endpoint control tests one nearby alternative, but a matched compact per-frame predictor is still missing; token counts alone cannot establish superiority over CompACT.

Steering policy proposals. Generative Predictive Control forecasts action consequences and uses rewards to rank or refine policy proposals [13]. FOREWARN aligns latent prediction with visual-language reasoning [15]; DreamSteer evaluates predicted visual progress with a language-conditioned critic [4]. Robot Critics trains fine-grained comparisons on paired actual success/failure rollouts and applies its critic to generated consequences [14]. V-GPS provides direct offline-RL value guidance [11]. Thus, proposal-based planning, goal-independent dynamics, and scoring predicted outcomes without reading actions all have precedents. CTA changes the *learned prediction target* in that pipeline. Our direct baseline is a controlled geometry-ranking model, not a reproduction of V-GPS, and the endpoint baseline is not a reproduction of GPC.

Conditional temporal codes. TAP already learns state-conditioned discrete trajectory codes and plans in their latent action space [7]. CTA predicts a code from an externally proposed action chunk rather than searching for actions through a trajectory decoder. Delta-IRIS uses context-aware temporal tokenization for efficient world modeling [10]; CTA

Approach	Predicted/searched object	Relationship to CTA
DINO-WM	Spatial visual features	Larger visual target
CompACT	Compact frame tokens	Compactness precedent
GPC	Visual consequences	Outcome-scoring precedent
DreamSteer		
TAP	Conditional trajectory codes	Code-search rather than supplied-action forecast
Delta-IRIS	Context-aware step tokens	Conditional coding precedent
V-GPS	Action-conditioned value	Direct guidance alternative
CTA	Conditional segment code	Future-supervised target, forecast and reader

Table 1. Architectural positioning. CTA combines segment-level source learning with forecasting from supplied action proposals; performance is evaluated separately.

uses a multi-step target and a learned reader. Finite scalar quantization (FSQ) supplies the quantizer [9]. These precedents narrow the novelty claim to the segment-target training and deployment interface, with its measured decision consequences.

Decision-aware model learning. Value equivalence formalizes preservation of relevant value computations [5], and value-equivalent sampling studies lossy environment modeling under a value-based criterion [1]. TD-MPC2 also trains a control-oriented latent model [6]. These methods establish decision-aware modeling as a broader principle. CTA instantiates it with supervised candidate ranking and visual reconstruction of a finite segment target, evaluated empirically rather than through a value-equivalence or rate–distortion guarantee.

3. Problem and Representation Contract

At decision time t , the observed context is C_t . A frozen policy proposes K action chunks $\{\mathbf{A}_t^{(k)}\}_{k=1}^K$, each executed for at most L steps. Let $\tau_t^{(k)}$ denote the corresponding observed future segment available during training, and $\ell_t^{(k)}(g)$ a training utility label for goal g . The selector chooses an index and replans after execution. Offline ranking holds the context and all candidate chunks fixed; closed-loop selectors subsequently visit different states and therefore need not see identical future banks.

CTA imposes the following information flow:

$$S = q_\phi(C, \tau), \quad \hat{S} = h_\theta(C, \mathbf{A}), \quad s = D_\psi(C, \hat{S}, g). \quad (1)$$

The source encoder is a training-only module. The predictor receives neither a goal nor an actual future. The reader

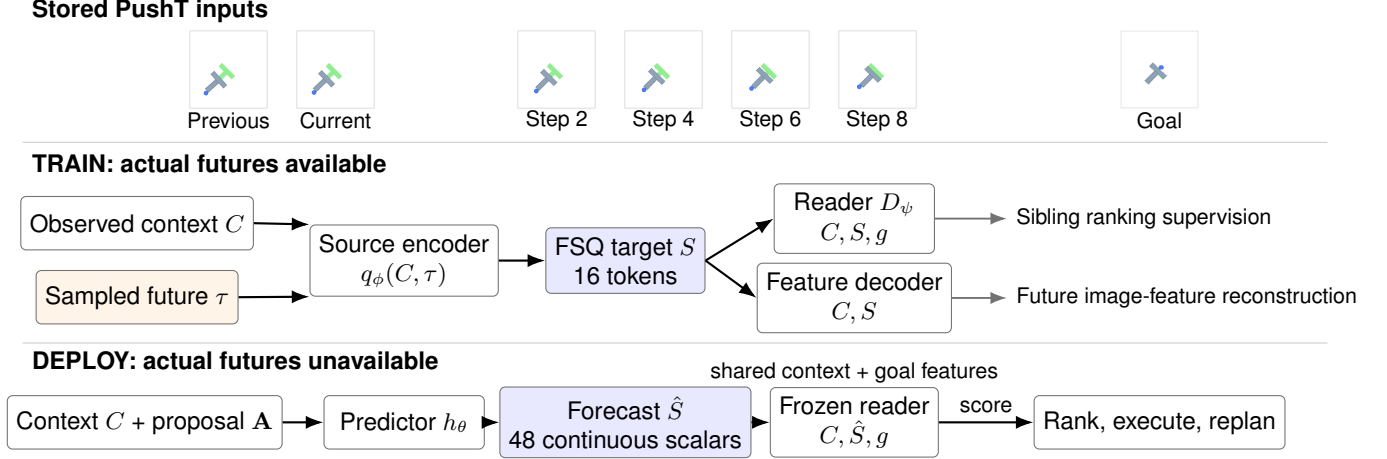


Figure 1. **CTA learns what the predictor must forecast.** Training uses observed context and sampled future observations to construct a quantized segment target. Ranking and feature reconstruction shape that target; the frozen reader then guides forecast training. Deployment uses context, policy proposals, and goal features. Only the predictor receives candidate actions; only training receives actual futures. The reader receives a continuous predictive mean. Images illustrate stored PushT observations, not a claimed successful deployment.

receives neither actions nor actual futures. Labels influence training losses but are not deployment inputs. Since context and goal are common across a bank, any candidate distinction must reach the reader through $\hat{S}$.

The deterministic, frozen source encoder produces $M = 16$ FSQ tokens with levels $(8, 8, 4)$, so the source alphabet has at most 256^{16} elements. This finite alphabet constrains source capacity; it does not measure the conditional entropy achieved by the encoder. Deployment uses the predictor’s coordinate means, $\hat{S} \in \mathbb{R}^{16 \times 3}$. These are *48 continuous scalars*, not a transmitted 128-bit code. The mean is a differentiable interface, not an identifiable uncertainty representation, and a nonlinear reader generally satisfies $D(C, \mathbb{E}[S], g) \neq \mathbb{E}[D(C, S, g)]$.

The design objective is to preserve candidate distinctions useful to the reader while reducing the target predicted by the dynamics model. Context conditioning allows information sharing but does not force residual-only coding. Intermediate future samples provide temporal supervision but do not prove intermediate-event retention. Similarly, withholding goals from the predictor permits reuse of its output, while transfer to new goal distributions remains an empirical question. Direct scoring and endpoint prediction are architectural comparators, rather than literal limiting cases of the finite source-code family.

4. Conditional Trajectory Abstraction

4.1. Inputs and learned source target

We use frozen DINOv2 ViT-S/14 features [12], projected to 128 dimensions using PCA fitted on training images.

Native 96×96 observations are resized to 224×224 . Context contains 256 current-image tokens, 64 pooled previous-image tokens, and one token embedding the current and previous agent positions divided by 512. An action a_h is embedded using its absolute and relative position, $[a_h/512 - 1/2, (a_h - p_t)/64]$.

The source future contains 256 endpoint-image tokens, 3×64 pooled intermediate-image tokens, and endpoint proprioception. Intermediate indices are $(2, 4, 6)$ for $L = 8$ and $(4, 8, 11)$ for $L = 15$. Segments stop at native termination or the episode limit; missing samples repeat the last observation. Thus τ summarizes selected observations, not every frame. Sixteen learned queries cross-attend to context and future tokens. FSQ maps their three-dimensional outputs to the grid $\mathcal{Q}_8 \times \mathcal{Q}_8 \times \mathcal{Q}_4$, where $\mathcal{Q}_l = \{(r - \lfloor l/2 \rfloor) / \lfloor l/2 \rfloor\}_{r=0}^{l-1}$.

4.2. Source reader and reconstruction

A Transformer reader $D_\psi(C, S, g)$ produces a scalar from context, code, and goal-image tokens. Training banks contain sibling futures from the same simulator state. With eligible ordered pairs $\mathcal{P} = \{(b, i, j) : \ell_{bi} - \ell_{bj} > \epsilon\}$, the ranking loss is

$$\mathcal{L}_{\text{rank}}(s, \ell) = \frac{1}{|\mathcal{P}|} \sum_{(b, i, j) \in \mathcal{P}} \text{softplus}[-(s_{bi} - s_{bj})]. \quad (2)$$

An empty pair set contributes zero. Ranking is within a bank; scores need not be comparable across contexts.

A feature decoder reconstructs endpoint and intermediate *image* features as residuals from the current full or pooled feature grid. It does not reconstruct proprioception. Endpoint

and intermediate MSEs are each divided by a train-set copy-current error estimate and averaged to form $\mathcal{L}_{\text{feat}}$. This auxiliary loss encourages future information; a context-dependent decoder can predict changes even with a constant code, so reconstruction is not a collapse-prevention guarantee. The source objective is

$$\mathcal{L}_{\text{source}} = \mathcal{L}_{\text{rank}} + \alpha \mathcal{L}_{\text{feat}} + \lambda_{\text{sat}} \mathbb{E}[(|z_{\text{pre}}| - 1.5)_+^2], \quad (3)$$

with straight-through quantization gradients. The last term discourages saturation of pre-quantized coordinates.

4.3. Parallel forecast and reader consistency

A Transformer encodes context and action tokens; M queries decode the segment target in parallel. Its output distribution factorizes over token coordinates:

$$p_{\theta}(S | C, \mathbf{A}) = \prod_{m=1}^M \prod_{j=1}^3 p_{\theta, m, j}(S_{mj} | C, \mathbf{A}), \quad (4)$$

$$\hat{S}_{mj} = \sum_{r=0}^{l_j-1} p_{\theta, m, j}(r | C, \mathbf{A}) \mathcal{Q}_{l_j}[r]. \quad (5)$$

Shared hidden computation does not remove the factorization of output probabilities. The categorical loss is the mean cross-entropy over $3M$ coordinates, $\mathcal{L}_{\text{NLL}}$. After source training, we freeze the encoder and reader and train only the predictor with

$$\mathcal{L}_{\text{pred}} = \mathcal{L}_{\text{NLL}} + \mathcal{L}_{\text{cons}} + \mathcal{L}_{\text{rank}}^w. \quad (6)$$

For a bank, predicted-reader scores s and source-reader scores u are centered by their own candidate means. We use

$$\mathcal{L}_{\text{cons}} = \text{SmoothL1} \left(\frac{s - \bar{s}}{T}, \frac{\text{sg}(u) - \bar{u}}{T} \right), \quad (7)$$

where T is a train-only RMS teacher-score scale, floored at 10^{-3} . Weighted ranking multiplies each eligible pair's term in Eq. (2) by $w_{ij} = \min(1, (\ell_i - \ell_j)/\sigma)$; its denominator remains the number of eligible pairs. Gradients through the frozen reader guide the forecast. Labels therefore supervise the predictor's losses even though they are not its inputs.

Centered consistency has a limited but useful interpretation. On a fixed bank, let $\varepsilon = \max_k |(s_k - \bar{s}) - (u_k - \bar{u})|$. If k_u and k_s maximize source and predicted scores, respectively, then $u_{k_u} - u_{k_s} \leq 2\varepsilon$. A unique source-score margin exceeding 2ε preserves its winner. The proof is in the supplement. This bounds reader-score regret, not native task regret: ranking scores are not calibrated physical utilities.

4.4. Training and deployment

Training executes proposals and perturbed proposals from cloned simulator states to collect sibling observations and

Algorithm 1 CTA deployment at one decision

Require: context C , cached goal features $\mathcal{G}$, policy π

- 1: $\{\mathbf{A}^{(k)}\}_{k=1}^K \leftarrow \pi(C)$ under the sampling protocol
- 2: $\hat{S}^{(k)} \leftarrow h_{\theta}(C, \mathbf{A}^{(k)})$ for all candidates
- 3: $s_k \leftarrow |\mathcal{G}|^{-1} \sum_{g \in \mathcal{G}} D_{\psi}(C, \hat{S}^{(k)}, g)$
- 4: $k^* \leftarrow$ smallest index maximizing s_k
- 5: execute up to L actions of $\mathbf{A}^{(k^*)}$; observe and replan

privileged geometry labels. All branches from an episode remain in one split. The dense label, measured at the executed chunk endpoint, is the negative mean distance between eight corresponding block vertices and goal vertices, divided by 512. This registration objective differs from PushT's native coverage-based success criterion. The protocol also collects states visited by an oracle-assisted selector to broaden training support.

The v2 recipe uses AdamW, learning rate 3×10^{-4} , weight decay .05, dropout .1, 16 decisions per update, 16,000 source-stage updates, and 10,000 prediction-stage updates. We use $\alpha = \lambda_{\text{sat}} = 1$, $\epsilon = .001$, and $\sigma = .01$. Model selection maximizes development retained geometry gap, averaging four fixed goal views; final scoring averages all sixteen views. These views depict one semantic target. The CTA codec/reader/decoder, FULL reader, and DIR scorer are separately parameterized but share a source-stage optimizer and global gradient clipping, coupling their optimization. The parameter-matched direct control is trained separately. Full configurations and selected checkpoints are provided in the supplement.

All modules have width 256 and eight attention heads. The source encoder has three decoder layers, the reader four encoder layers, the reconstruction decoder two decoder layers, and the predictor four encoder plus four decoder layers. No explicit conditional-entropy objective or context-only prior is trained in v2.

Context features and predicted codes can be cached across queries. A new raw goal image also requires feature extraction. A factorized direct scorer could likewise cache a goal-independent intermediate representation; that missing comparator limits attribution of query reuse to future supervision.

5. Experiments

5.1. Protocol, measurements, and scope

Task and data. PushT requires moving a planar T-shaped block into an image-specified pose. We use a frozen Diffusion Policy [3] as the proposal generator, $K = 8$, execution horizons $L \in \{8, 15\}$, and a 300-step episode limit. Native success requires post-action coverage greater than .95 before the limit. Training uses branched standard and perturbed proposals, with horizon-specific data mixtures. The principal

ranking test revisits 200 development initializations (roots 2200–2399), producing 5,558 P0-policy decision banks at $L = 8$; 3,433 have nonconstant geometry labels. Training and checkpoint-selection banks are separate from this evaluation population, but repeated development use means it is not a sealed test. All main models have one training seed. We report saved development evidence, not unrun multi-seed or held-out results.

Comparators. P0 always executes candidate zero. GEOM selects using privileged geometry labels; it is an unavailable deployment reference, not an upper bound on episode success. CTA predicts a code and uses its frozen reader. END predicts endpoint image features and proprioception with normalized MSE plus score consistency and weighted ranking. DIR reads context, actions, and goal directly and trains on the same dense labels. DIR-large raises the direct model to 14 layers and 11.31M parameters, compared with CTA’s 10.95M, END’s 11.14M, and DIR’s 6.57M. Counts exclude the frozen image encoder and policy.

We also adapt the official DINO-WM checkpoint [16] to score the same action banks. This comparison matches candidates, not all inputs or training: DINO-WM uses its own image encoding and current velocity, whereas our learned arms use position history and PCA features. Its training, precision, and checkpoint selection differ. We therefore treat it as an external reference, not a fully matched state-of-the-art claim. FULL reads actual endpoint futures; CODE reads source codes of actual sampled segments. Both use privileged future observations and are offline diagnostic tiers, not acting planners. FULL and CODE differ in reader/input structure, so their gap is not a pure compression ablation.

Ranking and uncertainty. For decision bank b , let y_{bk} be the geometry label and $\hat{k}_b$ the scorer’s choice. Retained gap is

$$\text{RG} = \frac{\sum_b (y_{b\hat{k}_b} - y_{b0})}{\sum_b (\max_k y_{bk} - y_{b0})}. \quad (8)$$

It measures recovered *aggregate geometry improvement* over candidate zero, not retained information or success probability. It can be negative; a zero denominator makes it undefined. Ties use the smallest candidate index. Saved 95% bootstrap intervals cluster decisions by initial root and use the same root resamples for paired contrasts. Closed-loop comparisons pair initial roots and report native episode success, paired percentage-point differences, and exact McNemar tests. These exploratory comparisons have no multiplicity correction and quantify episode uncertainty, not variation across training seeds.

5.2. RQ1: ranking on identical candidate banks

Figure 2 shows the strongest current evidence. CTA reaches RG .680 [.629,.726], versus END .635 [.588,.678], DIR .483

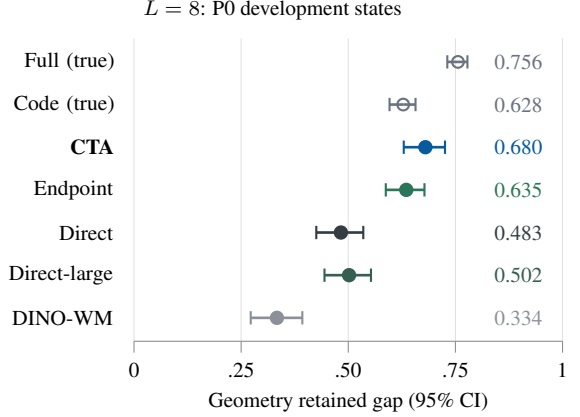


Figure 2. **Ranking on the same P0 decision banks.** $L = 8$, $K = 8$, 200 development roots. Horizontal bars are 95% root-bootstrap intervals. Hollow grey points use actual future information; filled points are deployable scorers. DINO-WM is an external checkpoint reference with unmatched observation/training details.

Paired RG difference	Estimate	95% CI
CTA – END	+.045	[.011,.079]
CTA – DIR	+.197	[.153,.241]
CTA – DIR-large	+.178	[.138,.220]
CTA – DINO-WM	+.346	[.294,.398]

Table 2. Paired root-bootstrap contrasts for Fig. 2. The last comparison retains the external-baseline caveats.

[.425,.535], and DIR-large .502 [.445,.553]. Paired differences in Tab. 2 support the CTA ranking advantage despite overlap between marginal intervals. Enlarging DIR improves its point estimate by .019, with interval $[-.021, .058]$; capacity alone does not explain the observed CTA–DIR difference. The control does not isolate future supervision from factorization, optimization, or regularization.

FULL reaches .756 [.731,.778], while CODE reaches .628 [.596,.657]. Predicted CTA exceeds CODE’s point estimate. Thus the diagnostic tiers are not a monotone oracle-to-code-to-prediction degradation ladder. In particular, the continuous predictive mean is a different reader input from a realized quantized source code; smoothing is a possible explanation, not a demonstrated mechanism. These diagnostics locate a discrepancy for further study rather than proving that prediction recovers information lost by compression.

5.3. RQ2: closed-loop behavior and computation

Table 3 reports historical runs using the old batched proposal sampler. Within each run, arms share the policy, candidate count, horizon, and seed mapping. Their visited states diverge after different selections. CTA succeeds in 140/200 episodes at $L = 8$, versus 138 for END, 135 for DIR, and 136 for DINO-WM. The paired learned-arm differences have

Arm	$L = 8$		$L = 15$	
	Success	Score	Success	Score
P0	122/200	.961	103/200	.911
GEOM (privileged)	160/200	.970	149/200	.948
CTA	140/200	.960	121/200	.930
END	138/200	.956	108/200	.936
DIR	135/200	.973	103/200	.920
DINO-WM	136/200	.974	112/200	.932

Table 3. **Historical development control, old proposal protocol.** One training seed; same 200 development roots at each horizon. Score uses maximum post-action coverage (excluding reset coverage), divided by .95 and clipped at one. Numerical sensitivity in Sec. 5.5 limits reproducibility. Horizon training mixtures differ, so this is not a pure horizon ablation.

Paired success difference	$L = 8$	$L = 15$
CTA – P0	+9 [1,17.5]	+9 [1,17]
CTA – END	+1 [−7.5,9.5]	+6.5 [−2.5,15.5]
CTA – DIR	+2.5 [−5.5,10.5]	+9 [.5,17.5]
CTA – DINO-WM	+2 [−6,10]	+4.5 [−4.5,13.5]

Table 4. Percentage-point differences and paired 95% root-bootstrap intervals for Tab. 3. Exact McNemar p for CTA–DIR at $L = 15$ is .05045 despite the bootstrap interval excluding zero; no unqualified significance claim is made. Complete p values are in the supplement.

broad intervals crossing zero (Tab. 4). CTA’s +9-point difference over P0 is more promising, but cannot establish superiority over the learned alternatives.

At $L = 15$, CTA reaches 121 successes compared with DIR’s 103. However, the change in their success difference between horizons is +6.5 points with interval [−4,17], and training data also change. This does not establish horizon robustness. Mean-score contrast intervals cross zero, and CTA does not lead the $L = 8$ mean-score column. A binary success gain and improved average coverage are distinct claims.

Component timing. Table 5 reports archived synchronized measurements on an H100 3g.40gb MIG instance: ten initial states, three warmups and ten repeats per state for scorer/context timing (one warmup and three repeats for policy sampling). CTA’s predictor/reader takes 4.11ms for one goal view and 16.87ms for sixteen. The latter are views of one target, not distinct semantic tasks. For our arms, the table excludes shared encoding; DINO-WM includes its own encoding, rollout, and cost. Precision is not fully matched. Therefore 4.11 versus 28.96ms is not a whole-planner speedup. The common policy sampler takes approximately 765ms and dominates this workload. End-to-end latency, memory, and matched-precision Pareto comparisons

Component ($K = 8$), median ms	$G = 1$	$G = 16$
Shared DINOv2/PCA context	6.49	6.48
CTA predictor + reader	4.11	16.87
END predictor + reader	4.52	23.40
DIR scorer	2.54	26.81
DINO-WM own encode + rollout + cost	28.96	29.07
Common policy sampling	765.33	765.33

Table 5. Component measurements with deliberately explicit scope. Our scorer rows exclude the shared context encoder and all rows exclude simulator execution. Rows with different scopes must not be interpreted as total-planner comparisons.

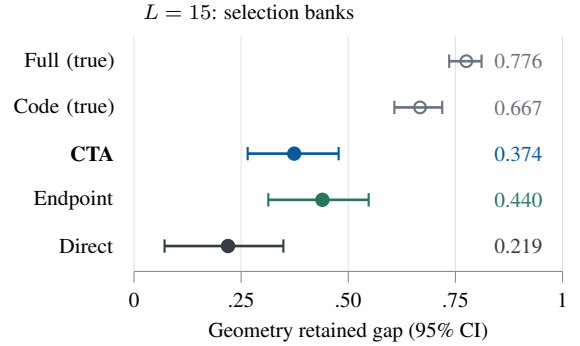


Figure 3. **A prediction bottleneck on $L = 15$ selection data.** $K = 8$, 1,476 banks, 100 roots (2000–2099). Source CODE retains substantially more geometry improvement than predicted CTA. This is a development diagnostic, separate from the P0 evaluation population in Fig. 2.

remain unmeasured.

5.4. RQ3: source, prediction, and objective failures

Longer-horizon prediction gap. On 1,476 selection banks from 100 separate development roots at $L = 15$, CODE achieves .667 [.608,.719] but predicted CTA reaches .374 [.265,.478] (Fig. 3). The .294 point difference identifies prediction/readout as a concrete bottleneck for this configuration. It is not a paired-confidence result, and selection data cannot provide an independent generalization test. The $L = 8$ selection population has a different mixture and should not be used as a matched horizon comparison.

Rare native-success crossings. Crossing banks, where candidate zero fails within the chunk but at least one sibling succeeds, are rare on logged P0 states: 58/5,558 at $L = 8$ and 35/3,249 at $L = 15$. CTA selects a successful candidate in .793 and .371 of these banks, respectively; source CODE captures .897 and .914. The geometry oracle captures .983 and 1.000, so its endpoint proxy is informative on these crossings. The weaker predicted capture points to forecast/readout fidelity; different objective definitions alone

do not establish widespread utility mismatch. Small denominators and downstream recovery limit translation to episode success. Ranking can improve many decisions without improving the few that determine termination. A native-hit continuation with matched CTA/END/DIR controls has been prepared, but no trained checkpoint or result is included here.

Predictability can destroy distinctions. An earlier, separate co-design pilot penalized source codes that were difficult for a predictor copy, frozen within each source-update block, to predict; source and predictor updates alternated. Increasing its weight from zero to .1 reduced prediction cross-entropy from .558 to .086 nats/token, but source RG fell from .706 to .458 (paired difference $-.248 [-.297, -.200]$). Token perplexity fell from 131.2 to 1.37 and sibling distinctness from .765 to .206. This is evidence of collapse in the tested pilot, not a universal rejection of co-design. It demonstrates why low forecast loss alone is insufficient evidence of a useful decision target. The pilot uses an earlier model and data regime, not the final v2 control comparison.

5.5. Numerical reproducibility and validity

The old proposal protocol changes sampled chunks with batch grouping. With the same 200 $L = 15$ roots and weights, two evaluations report P0 successes 100 and 103, but 43 roots change success status; GEOM has 29 such flips. Net counts therefore hide large episode-level disagreement. On one inspected root, grouping the policy batch from one to four changes initial actions by up to .0563 world units. This demonstrates one source of sensitivity, without fully attributing every discrepancy.

Canonical policy sampling fixes denoising microbatch shapes and candidate-prefix construction. A bounded P0 check for root 2205 alone versus grouped with root 2206 passes: $K = 1, 8, 16$ action prefixes match exactly, discrete decision fields agree, and coverage differs by at most 3.33×10^{-16} under an absolute tolerance of 10^{-12} . Scorer-shape handling is prepared in the continuation source but remains unqualified; the check has not certified every learned arm, root, or backend. Canonical samples also differ from historical ones. Consequently, Tab. 3 remains an explicitly historical result; it is not relabeled as a validated canonical evaluation. The next control result must be collected under the qualified protocol rather than inferred from archived counts.

6. Discussion and Conclusion

CTA investigates the representation a visual planner should predict when selecting among action chunks. A future-supervised conditional segment code improves geometry ranking on identical development banks, including over a parameter-matched direct scorer. This supports the proposed

target as a useful design direction. It does not yet show that conditioning, intermediate samples, quantization, or future supervision independently causes the gain: equal-capacity conditionality/path controls and a factorized direct comparator are missing.

The evidence also exposes where the current pipeline falls short. The $L = 15$ configuration shows a large source-to-prediction gap; dense geometry supervision and native success have different definitions, although the geometry oracle captures nearly all observed crossings; rare decisive events warrant focused forecast evaluation; and numerical proposal changes can alter episode outcomes. These findings motivate focused improvements in predictor/readout training, native-objective alignment, and deterministic evaluation. Their relative importance should be diagnosed within each configuration.

Practical impact depends on more than representation size. CTA uses privileged simulator labels and branched futures during training, a large frozen visual encoder, and a competent pretrained proposal policy. It cannot select an action absent from the proposal bank. Its 48-scalar deployment interface is compact, but its reader still processes context and goal tokens. Policy sampling dominates the measured workload; representation reduction is not equivalent to total compute reduction. Goal views share one PushT pose, so broad task or goal transfer remains untested.

The present conclusions are limited to reused development roots and one training seed. Historical control comparisons do not establish a reproducible learned-arm advantage, and no sealed multi-seed benchmark is claimed. The highest-priority validation is canonical closed-loop evaluation of matched continuation controls on independent roots and seeds, followed by comparable end-to-end timing and the ablations needed to attribute the ranking gain. Tasks with consequential intermediate events would separately test what the segment code preserves beyond endpoint ranking.

The central finding is therefore precise: learning a compact future-segment target can improve candidate ranking, but source quality, forecast fidelity, native-objective alignment, and reproducible control must be evaluated separately. Establishing their joint quality–compute benefit is the next step toward a stronger general planning contribution.

References

- [1] Dilip Arumugam and Benjamin Van Roy. Deciding what to model: Value-equivalent sampling for reinforcement learning. In *Advances in Neural Information Processing Systems*, pages 9024–9044, 2022. 1, 2
- [2] Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Mojtaba Komeili, Matthew Muckley, Ammar Rizvi, Claire Roberts, Koustuv Sinha, Artem Zhohus, Sergio Arnaud, Abha Gejji, Ada Martin, Francois Robert Hogan, Daniel Dugas, Piotr Bojanowski, Vasil Khalidov, Patrick

- Labatut, Francisco Massa, Marc Szafraniec, Kapil Krishnakumar, Yong Li, Xiaodong Ma, Sarath Chandar, Franziska Meier, Yann LeCun, Michael Rabbat, and Nicolas Ballas. V-JEPA 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025. 1, 2
- [3] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin CM Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Proceedings of Robotics: Science and Systems*, 2023. 4
- [4] Hanchen Cui, Sergio Arnaud, Arjun Majumdar, Daniel Dugas, Elie Aljalbout, Karthik Desingh, Krishna Murthy Jatavallabhula, and Franziska Meier. DREAMSTEER: Latent world models can steer VLA policies during deployment without any finetuning. *arXiv preprint arXiv:2607.02865*, 2026. 1, 2
- [5] Christopher Grimm, André Barreto, Satinder Singh, and David Silver. The value equivalence principle for model-based reinforcement learning. In *Advances in Neural Information Processing Systems*, pages 5541–5552, 2020. 1, 2
- [6] Nicklas Hansen, Hao Su, and Xiaolong Wang. TD-MPC2: Scalable, robust world models for continuous control. In *International Conference on Learning Representations*, 2024. 2
- [7] Zhengyao Jiang, Tianjun Zhang, Michael Janner, Yueying Li, Tim Rocktäschel, Edward Grefenstette, and Yuandong Tian. Efficient planning in a compact latent action space. In *International Conference on Learning Representations*, 2023. 1, 2
- [8] Dongwon Kim, Gawon Seo, Jinsung Lee, Minsu Cho, and Suha Kwak. Planning in 8 tokens: A compact discrete tokenizer for latent world model. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2026. 1, 2
- [9] Fabian Mentzer, David Minnen, Eirikur Agustsson, and Michael Tschannen. Finite scalar quantization: VQ-VAE made simple. In *International Conference on Learning Representations*, pages 51772–51783, 2024. 2
- [10] Vincent Micheli, Eloi Alonso, and François Fleuret. Efficient world models with context-aware tokenization. In *Proceedings of the 41st International Conference on Machine Learning*, pages 35623–35638, 2024. 1, 2
- [11] Mitsuhiro Nakamoto, Oier Mees, Aviral Kumar, and Sergey Levine. Steering your generalists: Improving robotic foundation models via value guidance. In *Proceedings of the 8th Conference on Robot Learning*, pages 4996–5013, 2025. 1, 2
- [12] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy V. Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, Mido Assran, Nicolas Ballas, Wojciech Galuba, Russell Howes, Po-Yao Huang, Shang-Wen Li, Ishan Misra, Michael Rabbat, Vasu Sharma, Gabriel Synnaeve, Hu Xu, Hervé Jégou, Julien Mairal, Patrick Labatut, Armand Joulin, and Piotr Bojanowski. DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research*, 2024. 2, 3
- [13] Han Qi, Haocheng Yin, Aris Zhu, Yilun Du, and Heng Yang. Inference-time enhancement of generative robot policies via predictive world modeling. *IEEE Robotics and Automation Letters*, 2025. Accepted for publication. 1, 2
- [14] Sruthi Sudhakar, Junbang Liang, Sreehari Rammohan, Pavel Tokmakov, Richard Zemel, and Carl Vondrick. Robot critics that sweat the small stuff. *arXiv preprint arXiv:2606.21572*, 2026. 2
- [15] Yilin Wu, Thomas Tian, Gokul Swamy, and Andrea Bajcsy. From foresight to forethought: VLM-in-the-loop policy steering via latent alignment. In *Proceedings of Robotics: Science and Systems*, 2025. 2
- [16] Gaoyue Zhou, Hengkai Pan, Yann LeCun, and Lerrel Pinto. DINO-WM: World models on pre-trained visual features enable zero-shot planning. In *Proceedings of the 42nd International Conference on Machine Learning*, pages 79115–79135, 2025. 1, 2, 5